ROOTCAUSE SERVER

15 · Riesgos y deuda técnica

Documento: 15-risks-and-technical-debt.md **Versión analizada:** 0.2.0 **Commit:** 767a67b **Fecha del análisis:** 2026-08-26

CONTENIDO

Escala usada

Resumen

R-01

El límite de 500 incidentes trunca en silencio postura, métricas y exportación

✅ Corregido

R-02

La ingesta de un sobre no es atómica

R-03

La identidad del activo colisiona con nombres de equipo repetidos

R-04

Un solo token, sin rotación ni caducidad

R-05

Datos personales permanentes dentro de los incidentes

R-06

Sin autorización ni RBAC

R-07

Rutas de lectura que recorren todos los incidentes y todos los activos

R-08

La superficie de seguridad no tiene historial

R-09

console/app.js: 1 177 líneas sin una sola prueba

R-10

Los parsers de salida de herramientas no tienen pruebas de robustez

R-11

Sin respaldo ni restauración automatizados

✅ Corregido

R-12

Sin registro de acceso autenticado exitoso

R-13

Sin cifrado en reposo

R-14

api.rs y storage.rs superan las 1 000 líneas

R-15

config/ existe en la copia de trabajo pero no en el repositorio

R-16

Features de tower-http declaradas y no usadas

R-17

El nombre del trabajo de cobertura no coincide con lo que mide

R-18

El estado del perímetro no sobrevive a un reinicio

R-19

Sin alta disponibilidad ni actualización sin interrupción

R-20

Sin cola local en el agente: los ciclos perdidos no se reenvían

R-21

La exención de RUSTSEC-2023-0071 ya no encuentra a qué aplicarse

Lo que se buscó y no se encontró
Decisiones que requieren validación humana

> Registro de hallazgos de este análisis. **Documento informativo: no se corrigió nada.** Cada entrada lleva severidad, impacto, probabilidad, evidencia verificable, ubicación, recomendación y prioridad.
>
> Muchos de estos puntos ya están declarados por el propio proyecto en `docs/CAPABILITIES.md` y `docs/THREAT_MODEL.md`. Se recogen igualmente, marcando cuáles son **límites conocidos y aceptados** y cuáles son **hallazgos nuevos** de esta revisión.

Escala usada

Severidad	Significado
Crítica	Puede producir un fallo de seguridad o una pérdida de datos
Alta	Afecta la corrección o la fiabilidad en uso normal
Media	Degrada la calidad, el rendimiento o el mantenimiento
Baja	Ruido, inconsistencia menor o mejora deseable

Prioridad	Significado
P1	Antes de la próxima versión
P2	En la versión siguiente
P3	Cuando toque la zona

Resumen

#	Hallazgo	Severidad	Prioridad	Tipo
R-01	El límite de 500 incidentes trunca en silencio postura, métricas y exportación	Alta	**P1**	**Corregido**
R-02	La ingesta de un sobre no es atómica	Alta	P2	Nuevo
R-03	La identidad del activo colisiona con nombres de equipo repetidos	Alta	P2	Nuevo
R-04	Un solo token, sin rotación ni caducidad	Alta	P2	Límite conocido
R-05	Datos personales permanentes dentro de los incidentes	Media	P2	Nuevo
R-06	Sin autorización ni RBAC	Media	P3	Límite conocido
R-07	Rutas de lectura que recorren todos los incidentes y todos los activos	Media	P2	Nuevo
R-08	La superficie de seguridad no tiene historial	Media	P3	Nuevo
R-09	`console/app.js`: 1 177 líneas sin una sola prueba	Media	P2	Nuevo
R-10	Los parsers de salida de herramientas no tienen pruebas de robustez	Media	P2	Nuevo
R-11	Sin respaldo ni restauración automatizados	Media	**P1**	**Corregido**
R-12	Sin registro de acceso autenticado exitoso	Media	P3	Nuevo
R-13	Sin cifrado en reposo	Media	P3	Límite conocido
R-14	`api.rs` y `storage.rs` superan las 1 000 líneas	Baja	P3	Nuevo

#	Hallazgo	Severidad	Prioridad	Tipo
R-15	`config/` existe en la copia de trabajo pero no en el repositorio	Baja	P3	Nuevo
R-16	Features de `tower-http` declaradas y no usadas	Baja	P3	Nuevo
R-17	El nombre del trabajo de cobertura no coincide con lo que mide	Baja	P3	Nuevo
R-18	El estado del perímetro no sobrevive a un reinicio	Baja	—	**Decisión deliberada**
R-19	Sin alta disponibilidad ni actualización sin interrupción	Baja	P3	Límite conocido
R-20	Sin cola local en el agente: los ciclos perdidos no se reenvían	Baja	P3	Nuevo
R-21	La exención de RUSTSEC-2023-0071 ya no encuentra a qué aplicarse	Baja	P3	**Nuevo**

R-01

El límite de 500 incidentes trunca en silencio postura, métricas y exportación

Campo	Valor
Severidad	Alta
Impacto	Una puntuación de postura, un contador de métricas o un paquete de evidencia pueden calcularse sobre un subconjunto de los incidentes sin que nada lo indique
Probabilidad	Media. Requiere superar 500 incidentes sin resolver o resueltos en la base
Ubicación	`storage.rs:504` (`list_incidents`), y sus llamadores en `api.rs`: `status`, `list_assets`, `topology`, `export`, `metrics`

Evidencia. `list_incidents` aplica siempre un `LIMIT`:

```
query = query.bind(filter.limit.unwrap_or(500).clamp(1, 5_000));
```

Cinco rutas lo invocan con `IncidentFilter::default()`, es decir sin `limit`, y por tanto con el tope de 500:

- `status` → `posture::compute_now(&incidents, None)` para toda la flota;
- `list_assets` → agrupa incidentes por activo para la postura individual;
- `topology` → riesgo y conteo por nodo;
- `export` → el **paquete de evidencia**;
- `metrics` → `rootcause_incidents_by_category` y `rootcause_posture_score`.

Además, el orden es por severidad descendente, así que lo truncado son los hallazgos **menos graves**. Eso limita el daño en la postura, pero no lo elimina: la puntuación por dimensión de las categorías con muchos hallazgos leves quedaría inflada, y en la exportación se pierden incidentes sin aviso.

Por qué importa especialmente en este producto. Es la misma clase de fallo que REQ-SEC-003 existe para prevenir —presentar un número calculado sobre menos datos de los que hay, sin declararlo—, aplicada a un camino distinto del que el requisito cubre.

Recomendación. Tres opciones, de menor a mayor esfuerzo:

1. Que `export` y los cálculos de postura y métricas usen un límite explícito y alto, o paginen hasta agotar.

2. Que `list_incidents` devuelva también el total, y que quien calcula un agregado declare si hubo truncamiento (coherente con `uninspected_surfaces`).
3. Calcular postura y contadores por categoría con `COUNT` / `GROUP BY` en SQL en lugar de materializando incidentes.

✅ Corregido

Se aplicó la tercera opción, que es la que elimina el problema en vez de desplazarlo:

- `IncidentFilter::limit` **significa lo que dice.** `None` es *sin límite*. Solo `GET /api/v1/incidents` fija un tope explícito (`INCIDENT_PAGE = 500`, ajustable con `?limit=`), porque es lo único que pinta una página para una persona.
- **La postura se calcula desde un resumen, no desde una página.** `posture::FindingCounts` y `posture::compute_from_counts` puntúan a partir de cuántos hallazgos abiertos hay de cada categoría y severidad; `Database::open_finding_counts` y `open_findings_by_asset` producen ese resumen con `COUNT(*)` y `GROUP BY` sobre **toda** la tabla.
- **El paquete de evidencia va completo**, incidentes y auditoría: `list_audit` acepta `None` para devolver el rastro entero.
- **Efecto secundario buscado:** `/api/v1/status`, `/metrics`, la topología y el inventario dejan de deserializar incidentes para contar, y `list_assets` ya no clona cada hallazgo. Eso reduce también R-07.

Pruebas que lo fijan: `an_aggregate_reads_the_whole_table_not_the_first_page` inserta 600 hallazgos y comprueba que el listado sin límite devuelve 600, que uno con `limit: Some(500)` devuelve 500 y que el resumen cuenta los 600; `a_summary_scores_exactly_like_the_incidents_it_summarises` verifica que contar en SQL da exactamente la misma puntuación que contar en memoria; `the_audit_trail_can_be_read_whole_or_paged` cubre la exportación completa.

R-02

La ingesta de un sobre no es atómica

Campo	Valor
Severidad	Alta
Impacto	Un fallo a mitad del ciclo deja escrituras parciales. El peor caso conocido: línea base de archivos actualizada sin haber guardado el incidente del cambio, lo que **pierde ese cambio para siempre**
Probabilidad	Baja. Exige caída del proceso o fallo de la base a mitad de la secuencia
Ubicación	`api.rs:229`, `ingest_telemetry`

Evidencia. El handler realiza entre 6 y 10 escrituras sucesivas sin envolverlas en una transacción. Solo `update_file_baseline` y `record_auth_pressure` abren transacción propia (`storage.rs:270` y `storage.rs:302`), y cada una cubre únicamente su bucle.

Mitigación existente. El orden está pensado: `update_file_baseline` va **después** de `upsert_incident`, de modo que una caída entre ambos deja el incidente guardado y la línea base sin actualizar —el ciclo siguiente volvería a detectar el mismo cambio, que es el desenlace preferible—.

Recomendación. Envolver el bloque de escritura del ciclo en una única transacción, o al menos el par `upsert_incident` + `update_file_baseline`. Añadir la prueba n.º 3 de la propuesta de 12.

R-03

La identidad del activo colisiona con nombres de equipo repetidos

Campo	Valor
Severidad	Alta
Impacto	Dos equipos con el mismo `hostname` comparten fila en `assets`, se pisan la telemetría y la superficie, y sus incidentes se mezclan bajo el mismo `asset_id`
Probabilidad	Media. Es habitual en autoescalado, plantillas de máquina virtual y contenedores
Ubicación	`main.rs:300`, `stable_agent_id`

Evidencia. El identificador es un UUID v5 derivado **solo** del nombre de equipo. La decisión tiene una razón buena —que el mismo equipo conserve su identidad entre reinicios sin guardar estado en disco—, pero el nombre de equipo no es único en muchos entornos reales.

Consecuencia observable: `upsert_asset` sobrescribe `hostname`, `platform` y `registration_json` en cada ciclo, así que la fila oscilaría entre los datos de un equipo y los del otro; `store_security` guardaría solo la última superficie recibida; y la regla de silencio nunca se dispararía mientras cualquiera de los dos reporte.

Recomendación. Documentar el requisito de unicidad del nombre de equipo, y ofrecer una opción `--agent-id` explícita o una semilla adicional (identificador de máquina, UUID persistido en disco) para entornos donde el nombre se repite.

R-04

Un solo token, sin rotación ni caducidad

Campo	Valor
Severidad	Alta
Impacto	Su compromiso da acceso total de lectura y la capacidad de publicar telemetría como cualquier agente
Probabilidad	Baja–media. Depende de la higiene operativa
Ubicación	`config.rs`, `auth.rs`
Estado	**Límite conocido**, declarado en `docs/CAPABILITIES.md`

Lo que sí está bien resuelto: entropía alta, comparación en tiempo constante, longitud mínima, rechazo de espacios envolventes, nunca se registra, oculto en la ayuda, bloqueo tras fallos repetidos.

Lo que falta: rotación con periodo de solapamiento —hoy cambiar el token obliga a reconfigurar servidor y todos los agentes a la vez, con interrupción—, caducidad, y separación entre credencial de lectura y credencial de ingesta.

Recomendación. Aceptar una lista de tokens válidos para permitir rotación sin interrupción, y considerar un token de solo ingesta para los agentes.

R-05

Datos personales permanentes dentro de los incidentes

Campo	Valor
Severidad	Media
Impacto	Direcciones IP y nombres de usuario quedan **indefinidamente** en `incidents.incident_json`, fuera del alcance de la retención
Probabilidad	Alta. Ocurre en cuanto se dispara cualquier regla de intrusión
Ubicación	`storage.rs:735` (`purge`), `incidents`

Evidencia. `purge` elimina de `telemetry`, `auth_pressure` y `defense_events`, pero **no** de `incidents` ni de `audit_log`. Los hallazgos de `intrusion.*` llevan la dirección de origen en el título, el resumen y la evidencia, y los nombres de usuario probados en la evidencia.

Tensión real: la decisión de no podar los incidentes es correcta —son la evidencia—. La consecuencia es que un requisito legal de supresión selectiva hoy solo se puede cumplir editando la base a mano.

Recomendación. Documentar la implicación en `docs/POLITICA_DE_PRIVACIDAD_LOCAL.md`, y valorar una operación de anonimización que sustituya una dirección concreta dentro del JSON conservando el resto de la evidencia. *Requiere validación jurídica según la jurisdicción del operador.*

R-06

Sin autorización ni RBAC

Campo	Valor
Severidad	Media
Impacto	Cualquier portador del token puede leer todo y cambiar el estado de cualquier incidente
Probabilidad	—
Ubicación	Todo el API
Estado	**Límite conocido y declarado**

Control compensatorio: el enlace en loopback por omisión y el proxy que controla el operador.

Recomendación. Mantenerlo fuera de alcance mientras el producto sea de un solo equipo, y anunciarlo con claridad si se plantea uso multiusuario.

R-07

Rutas de lectura que recorren todos los incidentes y todos los activos

Campo	Valor
Severidad	Media
Impacto	Latencia creciente con el tamaño de la flota y del histórico; presión de memoria por deserialización y clonado
Probabilidad	Media, con flotas grandes

Campo	Valor
Ubicación	`api.rs`: `status`, `list_assets`, `asset_detail`, `topology`, `metrics`, `export`

Evidencia.

- ~~`status`, `topology`, `metrics` y `export` cargan hasta 500 incidentes completos y los deserializan enteros solo para contar o agregar.~~ **Corregido junto con R-01:** las cuatro rutas usan ahora `open_finding_counts` / `open_findings_by_asset`, que cuentan en SQL.
- ~~`list_assets` llama a `group_incidents`, que **clona cada incidente** en un `HashMap<Uuid, Vec<Incident>>`.~~ **Corregido junto con R-01:** `group_incidents` ya no existe.
- `asset_detail` llama a `list_assets()` —toda la flota— y después busca uno con `position()`, en vez de consultar ese activo directamente.
- La consola llama a todas estas rutas **cada 10 segundos**.

Recomendación. Añadir a `Database` una consulta por identificador de activo; sustituir los conteos por `COUNT` / `GROUP BY` en SQL; evitar el clonado en `group_incidents` usando referencias. Añadir la prueba n.º 9 de 12.

R-08

La superficie de seguridad no tiene historial

Campo	Valor
Severidad	Media
Impacto	No se puede reconstruir qué puertos estaban abiertos en una fecha pasada, aunque sí queda el incidente que se generó entonces
Probabilidad	Alta cuando hace falta para una investigación
Ubicación	`storage.rs:225`, `store_security`

Evidencia. `store_security` hace `UPDATE assets SET security_json = ?`: solo existe la última superficie por activo. `telemetry` sí guarda historial de métricas, pero no de superficie.

Mitigación existente. Los incidentes conservan la evidencia del momento en que se dispararon, así que lo que produjo un hallazgo queda registrado. Lo que se pierde es el contexto no incidental.

Recomendación. Valorar una tabla `security_snapshots` sujeta a retención, o documentar explícitamente el límite en la vista de superficie.

R-09

`console/app.js`: 1 177 líneas sin una sola prueba

Campo	Valor
Severidad	Media
Impacto	Una regresión de renderizado no la detecta nada; la garantía de que no hay sumideros de marcado depende de una expresión regular

Campo	Valor
Probabilidad	Media
Ubicación	`console/app.js`

Evidencia. CI solo comprueba **sintaxis** (`node --check` en el trabajo «Consola embebida») y ejecuta `guard_console.py`, que busca patrones prohibidos. No hay ninguna prueba de comportamiento sobre las once funciones `render*`, sobre `layoutTopology` ni sobre el manejo de `401` y `429`.

Recomendación. Pruebas n.º 6 y n.º 7 de 12: un DOM sintético para las funciones de renderizado, y una prueba específica de que un nombre de equipo con marcado se muestra literal.

R-10

Los parsers de salida de herramientas no tienen pruebas de robustez

Campo	Valor
Severidad	Media
Impacto	Un formato inesperado de `ss`, `netstat`, `journalctl` o `wevtutil` puede producir una superficie incompleta —o silenciosamente vacía— sin declararla como brecha
Probabilidad	Media. Depende de versiones y localizaciones del sistema operativo
Ubicación	`net.rs`, `authlog.rs`, `baseline.rs`

Evidencia. Hay fixtures por plataforma y por herramienta, y están bien elegidas —incluyen estados en español —. Pero todas describen salidas *esperadas*. No hay pruebas con entrada arbitraria ni con formatos alterados.

Es la única superficie del sistema donde entra texto escrito por software de terceros. Un `parse_ss` que no reconociera ninguna línea devolvería una `NetworkSurface` vacía **sin brecha**, porque el comando sí tuvo éxito, y eso se leería como «este equipo no escucha en ningún puerto».

Atenuantes. Sin `unsafe`, sin aritmética que pueda desbordar sin control, y los parsers son funciones puras. Un fallo produce datos incompletos, no corrupción ni ejecución.

Recomendación. Prueba n.º 1 de 12 —la de mayor valor de todo el conjunto—. Y considerar emitir una brecha cuando un comando tiene éxito pero no produce **ninguna** fila reconocible.

R-11

Sin respaldo ni restauración automatizados

Campo	Valor
Severidad	Media
Impacto	Pérdida de incidentes y de auditoría, que **no se pueden regenerar**, ante un fallo de disco o un borrado
Probabilidad	Baja, con consecuencias altas
Ubicación	Ausencia en `scripts/`, `packaging/` y `docs/`

Evidencia. No hay script, tarea programada ni procedimiento documentado. El volumen `rootcause-data` de `compose.yml` preserva el archivo entre recreaciones del contenedor, pero un volumen no es un respaldo.

Recomendación. Añadir un `scripts/backup.sh` que use `sqlite3 .backup`, documentar la restauración y comprobarla al menos una vez. Es de las tareas de mejor relación valor/esfuerzo del repositorio.

✅ Corregido

`scripts/backup.sh` y su equivalente `scripts/backup.ps1`:

- usan `.backup` de SQLite, que es seguro con el servidor escribiendo, en vez de `cp` —que con WAL puede producir una instantánea rota si no viajan también los ficheros `-wal` y `-shm`—;
- **verifican la copia antes de darla por buena**: `PRAGMA integrity_check` más recuento de filas de `incidents`, `audit_log` y `assets` contra el origen. Si la copia no se puede leer, o tiene más filas que el origen, el script falla con código 1 en lugar de informar de un éxito;
- conservan las N copias más recientes (`ROOTCAUSE_BACKUP_KEEP`, 14 por omisión);
- imprimen el procedimiento de restauración, incluida la advertencia de que restaurar en un binario anterior al que creó el respaldo no está probado porque no hay migraciones descendentes.

Verificado contra una base creada con las migraciones reales del repositorio: copia, integridad, recuento de las tres tablas, retención, forma `sqlite://` de la ruta y el camino de fallo cuando la base no existe.

Lo que sigue siendo responsabilidad de quien opera: programar la ejecución (cron, temporizador de systemd o tarea programada), llevarse las copias fuera del anfitrión y **probar una restauración de verdad**. Un respaldo que nunca se restauró sigue sin ser un respaldo.

R-12

Sin registro de acceso autenticado exitoso

Campo	Valor
Severidad	Media
Impacto	Se sabe quién **falló** al autenticarse, no quién **leyó** la evidencia
Probabilidad	—
Ubicación	`auth.rs`

Evidencia. `record_failure` y `record_defense` cubren rechazos y bloqueos. `record_success` solo limpia el contador en memoria: no persiste nada. La auditoría cubre cambios de estado de incidentes, no lecturas.

Para un producto cuyo valor es la evidencia recuperable, saber quién descargó `GET /api/v1/export` tiene sentido.

Recomendación. Registrar los accesos a `export` y `audit` como entradas de auditoría, o al menos como línea `info` de `tracing`.

R-13

Sin cifrado en reposo

Campo	Valor
Severidad	Media
Impacto	Quien lea el archivo SQLite ve nombres de equipo, direcciones IP y nombres de usuario
Probabilidad	Baja, si los permisos son correctos
Ubicación	`storage.rs:80`
Estado	**Límite conocido**

Controles compensatorios: usuario dedicado en systemd y en el contenedor, `ProtectSystem=strict`, `ReadWritePaths` acotado, `read_only` en el contenedor.

Recomendación. Documentar que la confidencialidad en reposo depende del sistema de archivos, y recomendar cifrado de volumen en despliegues sensibles.

R-14

`api.rs` y `storage.rs` superan las 1 000 líneas

Campo	Valor
Severidad	Baja
Impacto	Coste de mantenimiento; `ingest_telemetry` concentra todo el ciclo en una función
Probabilidad	—
Ubicación	`api.rs` (1 100 líneas), `storage.rs` (1 159), `console/app.js` (1 177)

Atenuantes. Ambos archivos están bien seccionados con comentarios de bloque, y buena parte de su tamaño son pruebas dentro del mismo archivo, que es la convención de Rust.

Recomendación. Si `api.rs` sigue creciendo, separar por área (`api/incidents.rs`, `api/exposure.rs`…) y extraer las fases de `ingest_telemetry` en funciones con nombre.

R-15

`config/` existe en la copia de trabajo pero no en el repositorio

Campo	Valor
Severidad	Baja
Impacto	Confusión local: quien trabaje en esta copia puede creer que ahí va la configuración. Quien clone el repositorio no verá nada
Probabilidad	Baja
Ubicación	`config/`, solo en el árbol de trabajo

Evidencia. El directorio existe en disco y está vacío. `git ls-files config` devuelve **0 archivos** y no hay ningún commit que lo toque: como git no versiona directorios vacíos, `config/` nunca formó parte del repositorio. Una búsqueda de `config/` en el código, los scripts, los workflows, el `Dockerfile` y `compose.yml` tampoco devuelve ninguna referencia.

Recomendación. Borrarlo del árbol de trabajo, o —si la intención era ofrecer un lugar para la política de detección— crearlo de verdad con un `README.md` y una política de ejemplo, que sí quedarían versionados y encajarían con `--policy-file`.

R-16

Features de `tower-http` declaradas y no usadas

Campo	Valor
Severidad	Baja
Impacto	Superficie de dependencias mayor de la necesaria; contradice la disciplina del propio proyecto de desactivar features que sobran
Probabilidad	—
Ubicación	`Cargo.toml`

Evidencia. El workspace declara `features = ["compression-gzip", "cors", "limit", "request-id", "set-header", "timeout", "trace"]`. El router usa `CompressionLayer`, `RequestBodyLimitLayer`, `TimeoutLayer` y `TraceLayer`. Una búsqueda de `CorsLayer`, `RequestId` y `SetResponseHeader` en `crates/` no devuelve nada: **`cors`, `request-id` y `set-header` no se usan**.

Nota positiva: que CORS no esté montado es lo correcto para este producto. Lo que sobra es la feature, no la ausencia de la capa.

Recomendación. Quitar las tres features. Es un cambio de una línea que `cargo deny` y el SBOM agradecerán.

R-17

El nombre del trabajo de cobertura no coincide con lo que mide

Campo	Valor
Severidad	Baja
Impacto	Un lector puede creer que el umbral del 80 % se aplica solo al motor de detección
Probabilidad	Media
Ubicación	`.github/workflows/ci.yml:117`

Evidencia. El paso se llama «Medir cobertura del núcleo de detección», pero el comando es `cargo llvm-cov --workspace --locked --lcov --output-path lcov.info --fail-under-lines 80`: cubre **todo el workspace**.

Es exactamente el tipo de desincronización entre texto y código que `guard_claims.py` existe para evitar, en un lugar que ese guardián no revisa.

Recomendación. Renombrar el paso a «Cobertura del workspace, umbral 80 %».

R-18

El estado del perímetro no sobrevive a un reinicio

Campo	Valor
Severidad	Baja
Impacto	Reiniciar el proceso levanta todos los bloqueos en curso
Ubicación	`defense.rs`
Estado	**Decisión deliberada y documentada en el código**

El módulo lo justifica: el estado del perímetro debe sobrevivir a una ráfaga, no a un reinicio, y un reinicio es exactamente cuando un operador quiere empezar de cero. Se recoge aquí para que quede constancia, **no como defecto**.

Efecto secundario útil: es la vía de escape cuando un operador se bloquea a sí mismo. Ver 14 · Solución de problemas.

R-19

Sin alta disponibilidad ni actualización sin interrupción

Campo	Valor
Severidad	Baja
Impacto	Cada actualización deja un hueco en la telemetría
Ubicación	Arquitectura general
Estado	**Límite conocido y declarado**

SQLite no admite dos escritores concurrentes, así que dos instancias sobre el mismo archivo no son una opción. Los agentes reintentan solos, de modo que el efecto práctico es un hueco, no una pérdida de servicio.

R-20

Sin cola local en el agente: los ciclos perdidos no se reenvían

Campo	Valor
Severidad	Baja
Impacto	Un corte de red prolongado deja un hueco permanente en el historial de ese activo
Probabilidad	Media
Ubicación	`main.rs`, bucle principal

Evidencia. `send_once` construye la muestra en el momento del envío. Si falla, se registra la advertencia, se dobla el intervalo y el ciclo siguiente reporta el estado **actual**. Nada guarda lo no entregado.

Atenuante. Es coherente con el producto: RootCause responde «qué está pasando», y una muestra de hace veinte minutos ya no responde esa pregunta. El efecto es un hueco visible, no un dato erróneo, y la regla `availability.agent.silence` lo señala.

Recomendación. Documentar el comportamiento en el manual. Si en el futuro se quisiera cola, tendría que llevar tope de tamaño y no persistir en un disco que podría estar lleno —que es justo una de las condiciones que el producto detecta—.

R-21

La exención de RUSTSEC-2023-0071 ya no encuentra a qué aplicarse

Campo	Valor
Severidad	Baja
Impacto	Una exención muerta en la política de dependencias. No debilita nada hoy, pero es una línea que dice «aquí aceptamos un riesgo» cuando el riesgo ya no existe
Probabilidad	—
Ubicación	`deny.toml` línea 14, `.cargo/audit.toml`

Evidencia. Ejecutando `cargo deny --all-features check` en este análisis:

```
warning[advisory-not-detected]: advisory was not encountered
14 | ignore = ["RUSTSEC-2023-0071"]
   |           ━━━━━━━━━━━━━━━━━━ no crate matched advisory criteria
```

Y `cargo tree --invert rsa` responde `nothing to print`: **`rsa` no está en el grafo de dependencias**, ni siquiera con todas las features activadas.

Por qué esto es una buena noticia. La premisa de la exención era que `rsa` aparecía en `Cargo.lock` —un archivo agnóstico a las features— pero nunca se compilaba. Hoy ya ni siquiera aparece: el motivo desapareció por completo.

Matiz importante. El paso de CI que respalda la exención comprueba justamente que `cargo tree --invert rsa` no devuelva nada, así que **el control sigue haciendo su trabajo**. Lo que sobra es la exención, no la comprobación.

Recomendación. Retirar `RUSTSEC-2023-0071` de `deny.toml` y de `.cargo/audit.toml`, y **conservar** el paso de CI que verifica que `rsa` sigue fuera del grafo: si algún día volviera a entrar, el aviso debe saltar en vez de quedar silenciado por una exención heredada.

Lo que se buscó y no se encontró

Para que la ausencia de un hallazgo signifique algo, conviene decir qué se revisó:

Búsqueda	Resultado
`unsafe`	**Ninguno.** Prohibido por el workspace

Búsqueda	Resultado
`unwrap()` / `expect()` fuera de pruebas	**Ninguno en código de producción.** Los 77 usos del repositorio están todos en `tests/control_plane.rs` o en módulos `#[cfg(test)]`
`todo!()`, `unimplemented!()`, `dbg!()`	**Ninguno.** Clippy los deniega
Interpolación de datos en SQL	**Ninguna.** Todo va por parámetros enlazados
Secretos en el repositorio o en el histórico	**Ninguno.** `gitleaks` en CI, más revisión manual de `.env.example`, `compose.yml`, `packaging/` y los workflows
`innerHTML` o manejadores en línea en la consola	**Ninguno.** `guard_console.py` en verde
Código muerto o módulos huérfanos	**Ninguno.** El único caso, `rca.rs`, se eliminó en el commit `2efedb2`
Acciones de GitHub sin pinnear	**Ninguna.** `guard_claims.py` en verde
Dependencias con aviso sin exención justificada	**Ninguna.** `cargo audit --deny warnings`: 315 dependencias contra 1 226 avisos, sin hallazgos. `cargo deny --all-features check`: `advisories ok, bans ok, licenses ok, sources ok`
Cifras de la documentación desincronizadas del código	**Ninguna** en las que cubre `guard_claims.py` (18 reglas, versión `0.2.0`)
Archivos con BOM, CRLF o mojibake	**Ninguno.** 109 archivos verificados
Fallos de prueba	**Ninguno.** 253 pruebas, 0 fallos, el 2026-08-26

Decisiones que requieren validación humana

#	Decisión	Quién debe validarla
1	~~Si el límite de 500 incidentes es aceptable para la exportación de evidencia~~ — **resuelto: ya no hay límite en la exportación** (R-01)	—
2	Si el nombre de equipo basta como identidad de activo en su flota (R-03)	Quien opere
3	Retención adecuada al marco legal aplicable (R-05)	Legal y quien opere
4	Si hace falta cifrado en reposo en su despliegue (R-13)	Seguridad
5	El respaldo ya está automatizado y se verifica solo (R-11); falta **programarlo, sacarlo del anfitrión y probar una restauración real**	Quien opere
6	Que la atestación de procedencia de la publicación funciona (13)	Quien publique
7	Que el instalador de Windows produce un instalador funcional (13)	Quien publique

Anterior: 14 · Solución de problemas · **Siguiente:** 16 · Glosario · **Índice:** README